* An inner class represents a **strong encapsulation with outer class** because inside an inner class we can directly access the private data of outer class.

Example for a an ordinary class :

```
public class Trainer
{
   public static void viewStudentProfile(Student student)      // Student Object
   {
       int marks = student.getMarks();
   }
}

class Student
{
   private int marks;

   //Parameterized Constructor

   //setter (To modify the marks)

   //getter (To read the private data)

}
```

Example of an inner class :

```
class Outer
{
     private int x = 100;

     class Inner
     {
          public void show()
          {
               IO.println("Show method "+x);
          }
     }
}
```

Can access private data of Outer class without getter

* An inner class represents HAS-A realtion with outer class.

Example :

```
public class Outer
{
    class Inner  //Outer class HAS inner class   (HAS-A Realtion)
    {
    }
}
```

* If it represents HAS-A relation then when we should use HAS-A Relation and when we should use inner class ?

Case 1 :

Use HAS-A Relation if, we can use a **particular class** in multiple places as a property.

```
class Employee
{
  private Address address;

}

class Person
{
  private Address address;

}

class Student
{
  private Address address;

}

class Address
{

}
```

Case 2 :

We should use inner class when a class (inner class) is only meant for outer class but not for others.

Example 1

```
class Person
{

   class Heart
   {
   }
}
```

Example 2

```
class Laptop
{
   //switchOn()

  class Motherboard
  {
     //booting process
  }
}
```

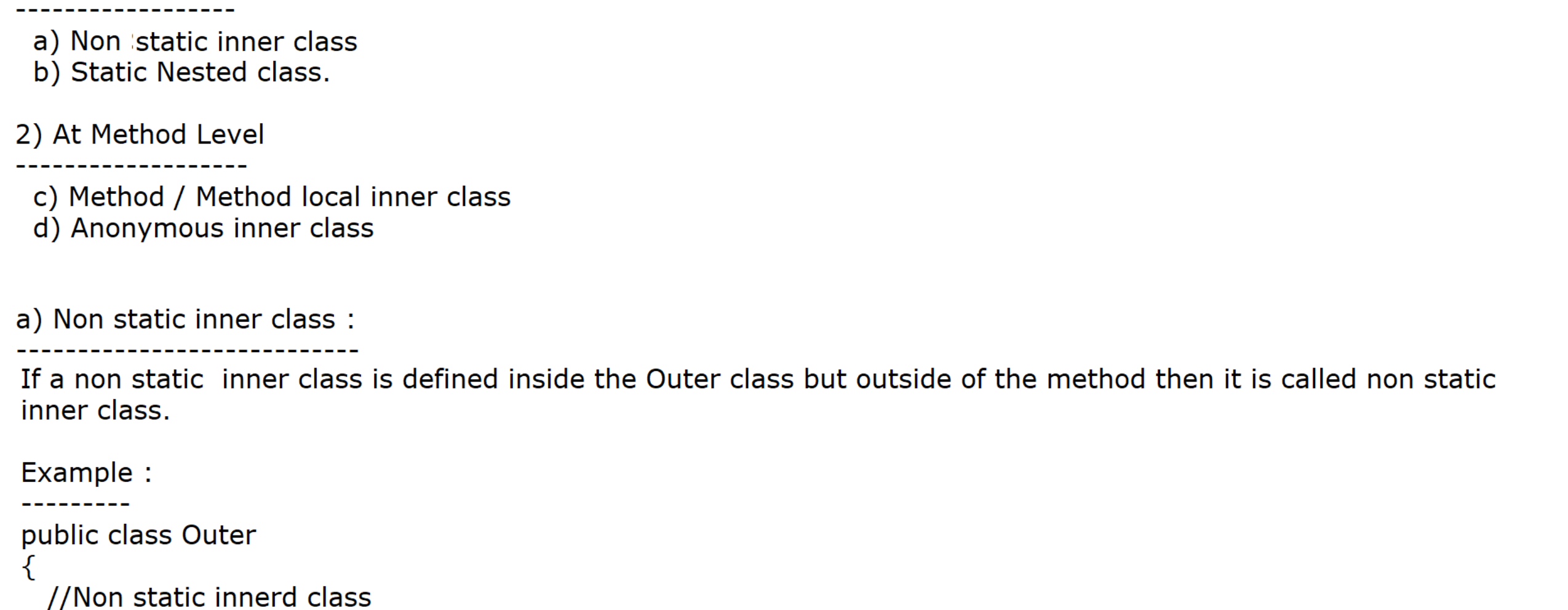
Example 3

```
class University
{

   class Dept
   {
   }

}
```

* An inner class is represented by $ symbol in .class file format.

Advantages of inner class :
----
1) It is a way of logically grouping classes that are only used in one place
2) Provides strong encapsulation with outer class.
3) It provides more readable and maintainable code. [Outer class code is isolated from inner class code]
4) Hiding the implementation [If inner class is declared as private]

Types of Nested/inner classes :



There are 4 types of inner classes in java, 2 inner classes we can write at class level (static + non static) and two inner classes we can write a method level (local + anonymous)

1) At Class Level
----
a) Non static inner class
b) Static Nested class.

2) At Method Level
----
c) Method / Method local inner class
d) Anonymous inner class

a) Non static inner class :

If a non static inner class is defined inside the Outer class but outside of the method then it is called non static inner class.

Example :

```
public class Outer
{
  //Non static innerd class
  private class Inner
  {
  }
}
```

In non static inner class we can apply private, default (private-package), protected, public, abstract and final modifiers.

During the class loading phase an inner class will be loaded with the help of Outer class. If inner class contains non static member

For non static inner class, Outer class object is required, We can't create the inner class object directly, Outer class object is required.

It is also known as Regular OR Member class.

//Programs

```
package com.ravi.non_static_inner_class;

class Outer
{
    private int x = 100;

    class Inner //Non static inner class
    {
        public void show()
        {
            //Can directly access the private data outer class (Encapsulation)
            IO.println("Outer class x variable is :"+x);
        }
    }
}

public class NonStaticInnerClassDemo1
{
    public static void main(String[] args)
    {
        Outer.Inner in = new Outer().new Inner();
        in.show();
    }
}
```

Variable Shadow in inner class :

If outer class and inner class variable names are same then inner class variable will be access with inner class object (Variable Shadow bacause in the same scope), If we want to access Outer class variable from inner class then we need **OuterClassName.this.variableName**.

```
package com.ravi.non_static_inner_class;

class MyOuter
{
    int x = 100;

    class Inner
    {
        int x = 200;

        public void show()
        {
            IO.println("inner class x variable value is :"+this.x);
            IO.println("Outer class x variable value is :"+MyOuter.this.x);
        }
    }
}

public class NonStaticInnerClassDemo2
{
    public static void main(String[] args)
    {
       MyOuter outer = new MyOuter();
       MyOuter.Inner in = outer.new Inner();
       in.show();

    }

}
```

```
package com.ravi.non_static_inner_class;

class Person
{
    private String name;
    private int age;
    private Heart heart;

    public Person(String name, int age)
    {
        this.name = name;
        this.age = age;
        this.heart = new Heart(72);
    }

    public void getPersonInfo()
    {
        IO.println("Name is :"+name);
        IO.println("Age is :"+age);
        IO.println("Heart beat is :"+heart.heartBeats);
    }

    private class Heart
    {
        private int heartBeats;

        public Heart(int heartBeats)
        {
           this.heartBeats = heartBeats;
        }
    }
}
public class NonStaticInnerClassDemo3
{
    public static void main(String[] args)
    {
        Person p = new Person("Scott", 23);
        p.getPersonInfo();

    }

}
```

```
package com.ravi.non_static_inner_class;

class University
{
    private String name;

    public University(String name)
     {
        this.name = name;
    }

    public void displayUniversityName()
     {
        IO.println("University Name: " + name);
    }

    public class Department
     {
        private String name;
        private String headOfDepartment;

        public Department(String name, String headOfDepartment)
          {
            this.name = name;
            this.headOfDepartment = headOfDepartment;
        }

        // Method to display department details
        public void displayDepartmentDetails()
          {
              displayUniversityName();
            IO.println("Department Name: " + name);
            IO.println("Head of Department: " + headOfDepartment);

        }
    }
}
public class NonStaticInnerClassDemo4
{
    public static void main(String[] args)
     {

        University university = new University("JNTU");

       University.Department cs = university.new Department("Computer Science", "Dr. John");

        University.Department ee = university.new Department("Electrical Engineering", "Dr. Scott");


        cs.displayDepartmentDetails();
        ee.displayDepartmentDetails();
    }
}
```